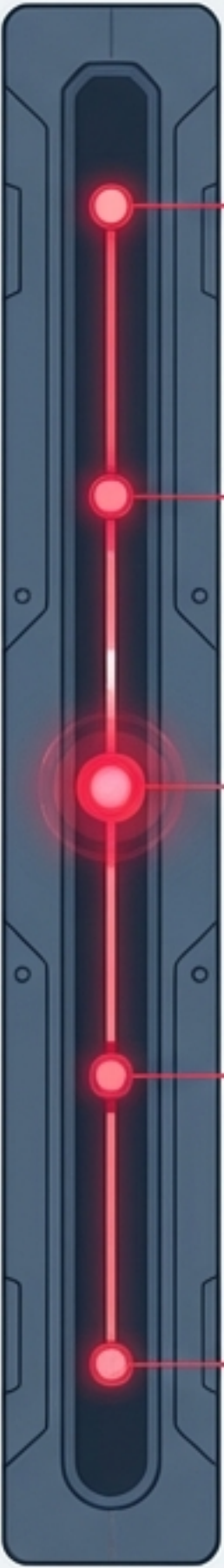


Introducción a Redis

Cache, estructuras en memoria y
patrones de uso en sistemas reales



1. El Paradigma:
Memoria vs. Disco

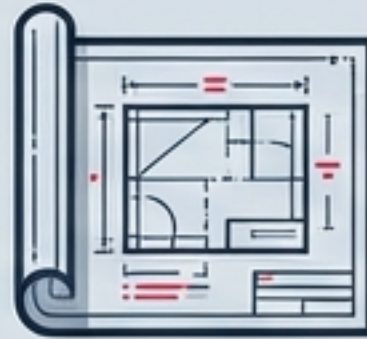
2. Catálogo de
Estructuras Nativas

3. Persistencia y
Contenedores (Docker)

4. Patrones de
Integración (Cache)

5. Mejores Prácticas
y Anti-patrones

Resultados Esperados



Arquitectura:
Explicar por
qué Redis
complementa
(no sustituye)
a PostgreSQL.



Ingeniería:
Elegir la
estructura de
datos correcta
para escenarios
 $O(1)$ u $O(\log n)$.



DevOps:
Operar Redis
en docker-
compose.yml
con
persistencia y
healthchecks.



Desarrollo:
Implementar
cache-aside y
write-through
evaluando sus
trade-offs.

PostgreSQL



La Verdad en Disco

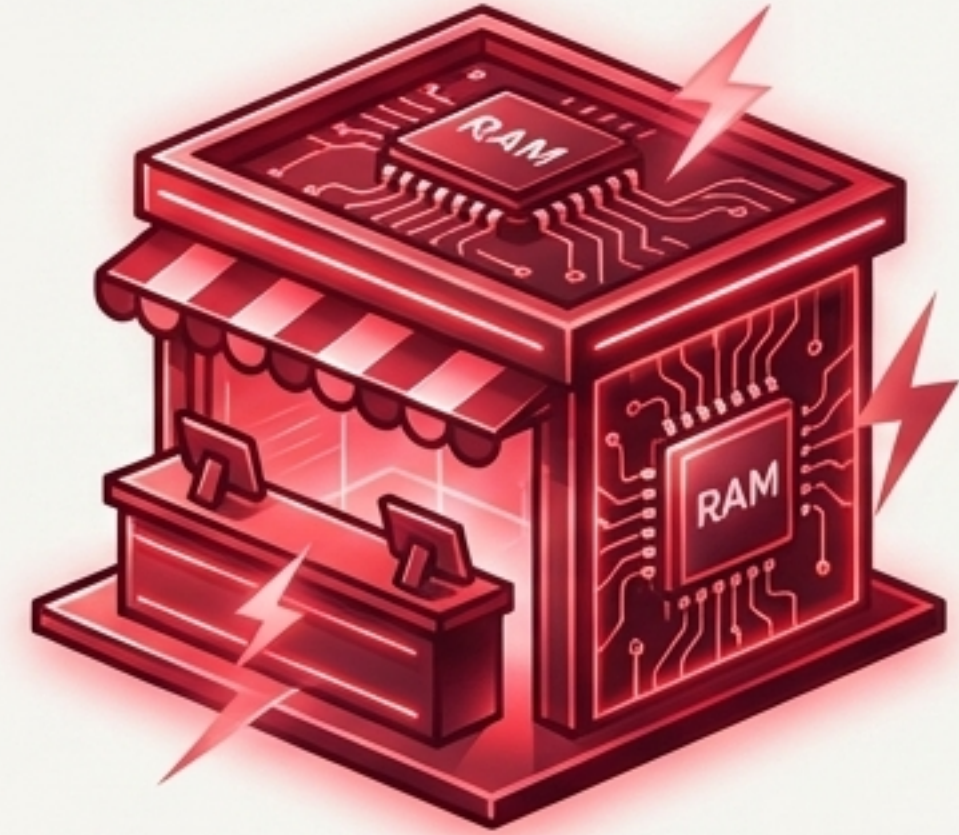
Datos seguros, organizados,
relacionales.

Operaciones pesadas.



El Límite: Nadie atiende clientes
desde la bodega.

Redis



La Verdad en Memoria

Single-threaded, clave-valor,
latencia sub-milisegundo.

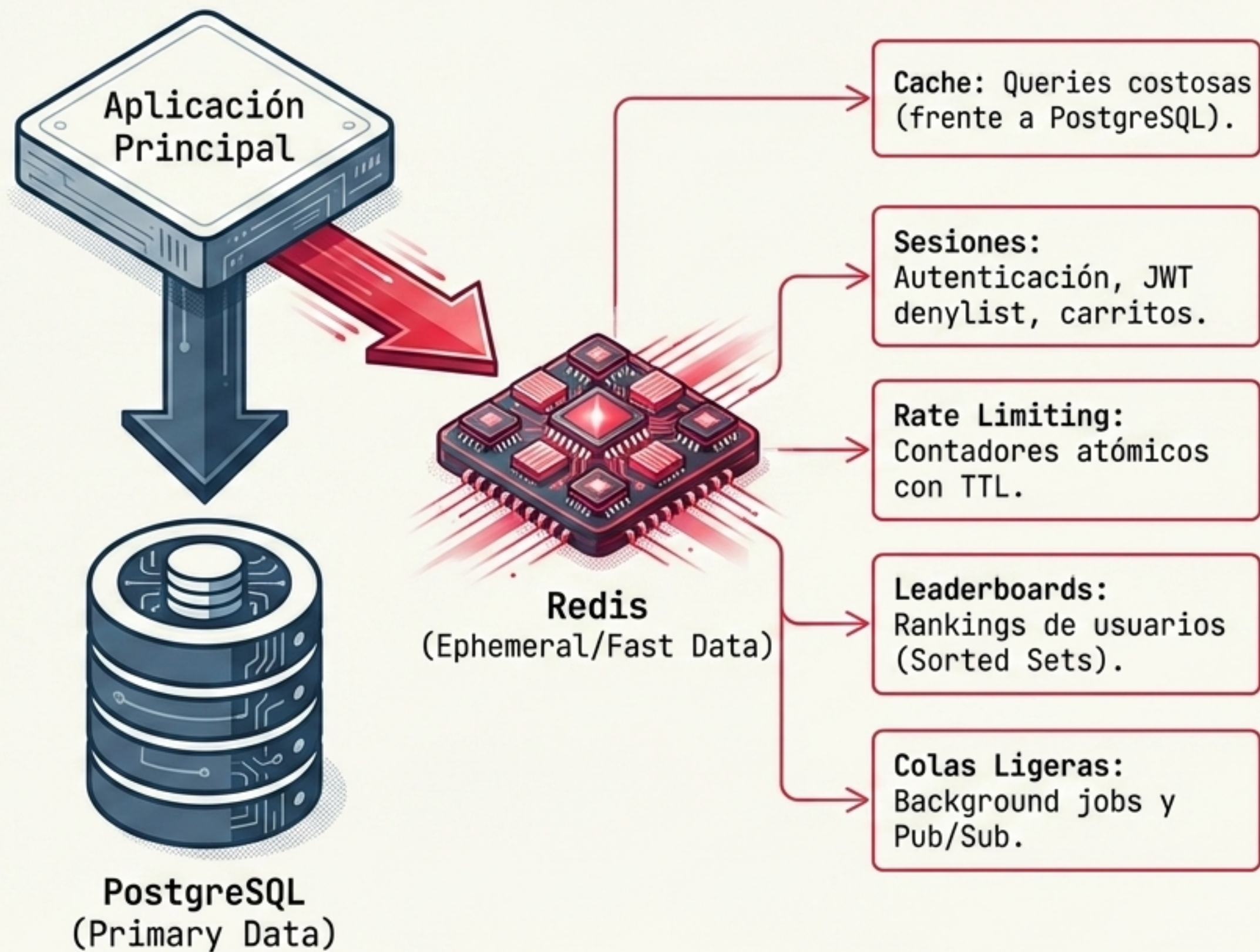
La Regla: Tu dataset debe caber en RAM.

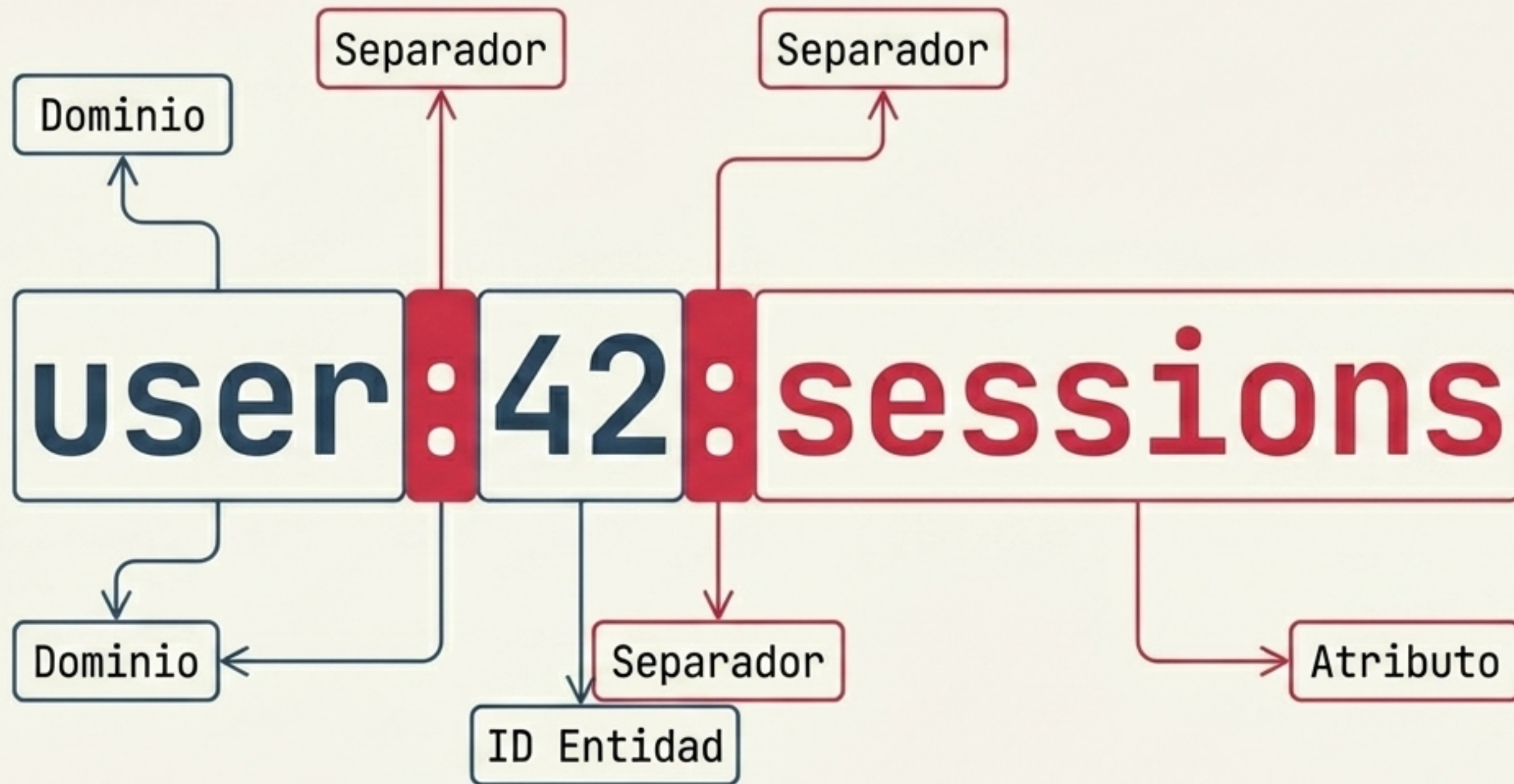


El Límite: Nadie hace inventario
general en el mostrador.

Caso Clínico: MAPS Disruptivo

Usa Redis para cachear embeddings frecuentes y mantener el estado de conversación de ~100 usuarios concurrentes sin asfixiar a PostgreSQL en cada turno.





Las Reglas de Oro

- No hay esquema interno; la convención es tu único contrato.
- **SÍ**: Usa ':' como jerarquía.
- **NO**: Nunca uses espacios o caracteres de escape.
- **OPTIMIZACIÓN**: Mantén claves legibles pero cortas (cada byte cuenta con millones de registros).
- **GOBIERNO**: Documenta tu convención en el repositorio como si fuera un schema SQL.

Strings

0(1)

- **Límite:** Hasta 512 MB por clave.
- **Casos de Uso:** JSON serializado, feature flags, contadores, sesiones con TTL.

```
SET session:abc 'user42' EX 3600  
INCR counter:visits  
TTL session:abc
```

Hashes

0(1)

- **Modelo Mental:** Como una fila SQL; muta columnas sueltas en 0(1) sin reescribir todo el JSON.
- **Casos de Uso:** Perfiles de usuario, objetos con actualizaciones parciales.

```
HSET user:42 name 'Ramsés' age 25  
HINCRBY user:42 age 1  
HGETALL user:42
```


Lists

Mecánica: Listas enlazadas dobles. $O(1)$ en extremos.

Uso: Colas productor/consumidor, feeds recientes.

```
LPUSH queue:emails 'e1'  
BRPOP queue:emails 5
```

Sets

Mecánica: Colecciones únicas, sin orden. Operaciones $O(N)$ en servidor.

Uso: Tags, deduplicación, permisos (SINTER).

```
SADD tags:1 'redis'  
SINTER tags:1 tags:2
```

Sorted Sets (La Estrella)



Mecánica: Elementos únicos con un Score (float) para auto-ordenamiento.

Uso: Leaderboards, rate limiting (sliding window), colas de prioridad.

```
ZADD leaderboard 1500  
'alice'  
ZREVRANGE leaderboard 0  
2 WITHSCORES
```


RDB - Redis Database

Mecánica: Snapshots binarios periódicos.

Pros: Archivo compacto, restauración ultra-rápida.

Contras: Pierdes datos desde el último snapshot ante un crash.



AOF - Append-Only File

Mecánica: Log de cada comando de escritura (fsync por segundo).

Pros: Alta durabilidad, pérdida casi nula.

Contras: Archivo pesado, restauración lenta.

El Estándar de Producción

Habilitar ambos. RDB para reinicios rápidos, AOF para evitar pérdida.
Nota: Para Cache Puro donde Postgres tiene la verdad absoluta, deshabilitar persistencia maximiza el rendimiento.

docker-compose.yml

```
services:
  redis:
    image: redis:7-alpine
    command: >
      redis-server --appendonly yes
      --maxmemory 256mb
      --maxmemory-policy allkeys-lru
    healthcheck:
      test: ['CMD', 'redis-cli', 'ping']
```

Imagen ligera (~40 MB).

Activa durabilidad AOF.

Crítico. Si se llena RAM, allkeys-lru evicciona claves viejas. Sin esto, Redis hace crash al límite.

Obliga a otros contenedores a esperar que Redis esté ready (**ping**), no solo encendido.

docker-compose.yml

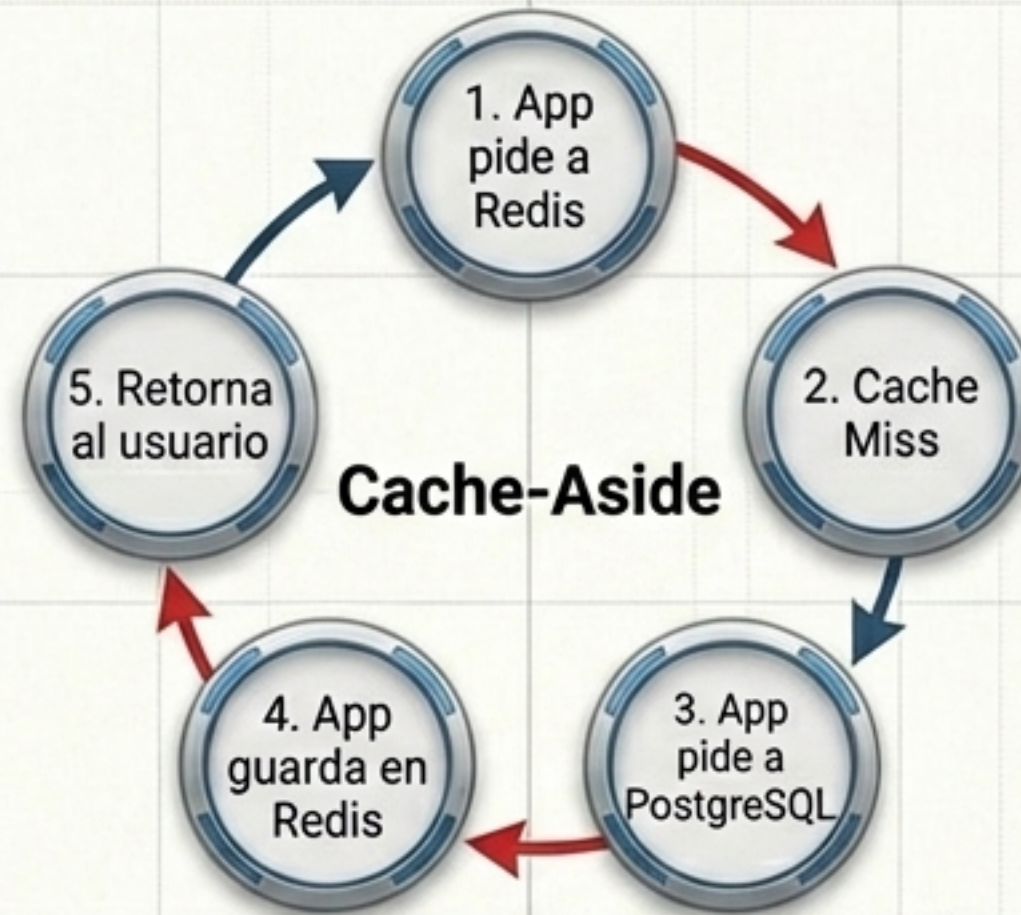
```
services:
  app:
    depends_on:
      redis:
        condition: service_healthy

    environment:
      REDIS_URL: redis://redis:6379/0
```

Evita errores de conexión al arrancar. La app no inicia hasta que Redis responda al PING.

El hostname es el nombre del servicio en Docker. ¡No uses localhost!

El número de base de datos lógica (0-15 por defecto). Idealmente, usa la 0 y despliega una instancia distinta por servicio en producción.



```
def get_product(sku):  
1 data = r.get(f'cache:product:{sku}')
```

2

```
2 if not data:  
3     data = db.query('SELECT...') # Fallback a PostgreSQL  
4     r.set(f'cache:product:{sku}', data, ex=300)  
return data
```

Trade-off Alert Widget

Lazy Loading: Robusto, pero el primer usuario sufre la latencia de Postgres. Peligro de Stale Data hasta que el TTL expire.



```
def update_price(sku, price):  
    db.execute('UPDATE...', price, sku)  
    r.delete(f'cache:product:{sku}')
```

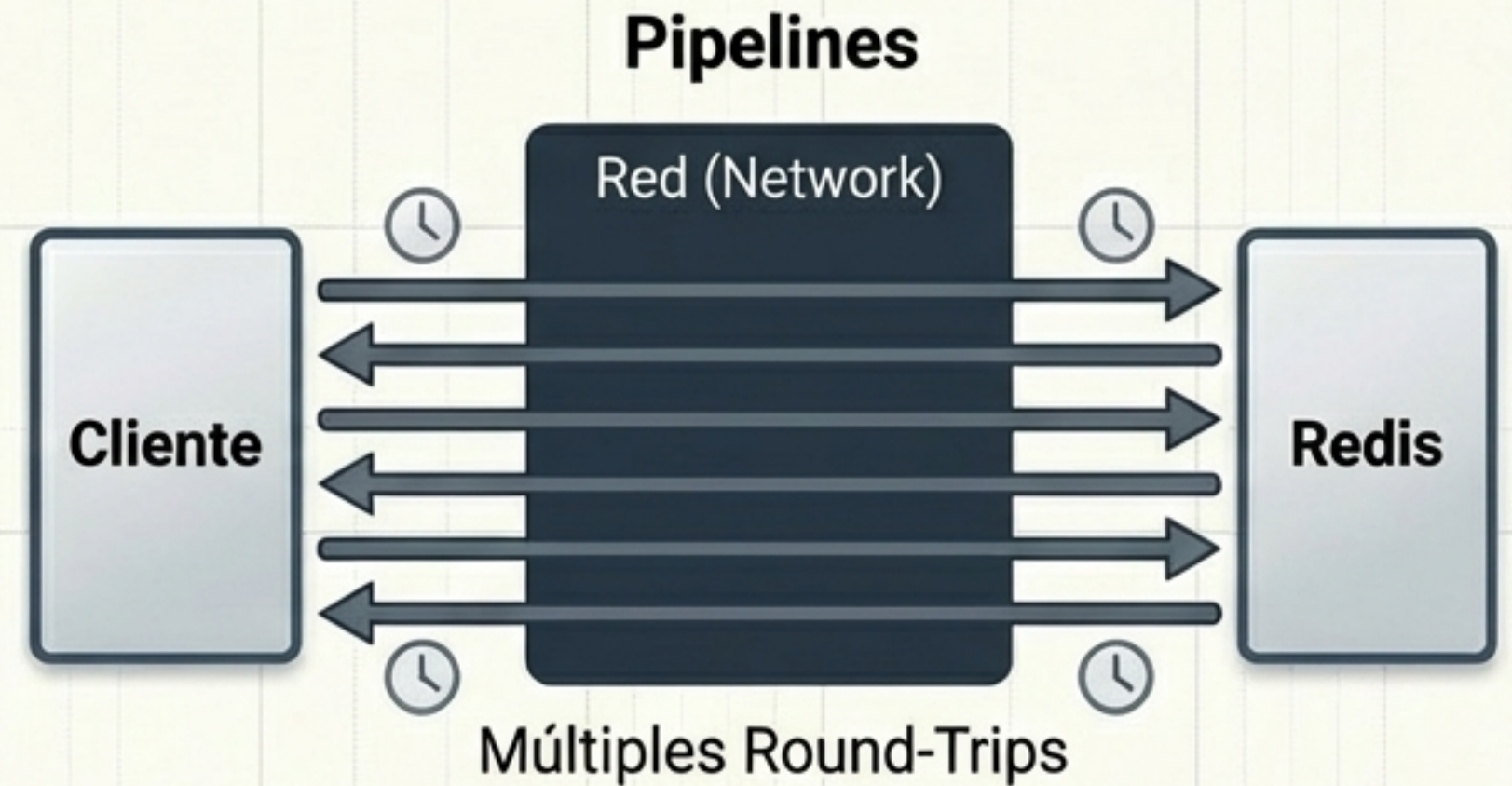
Trade-off Alert Widget



Riesgo de Dual-Write. Consistencia inmediata, pero frágil. Si la escritura a Postgres funciona, pero el DELETE a Redis falla, el sistema queda permanentemente inconsistente (requiere mitigaciones avanzadas).

El Check-list de Producción (Mejores Prácticas)

- ✓ **Siempre usa TTL:** Una clave sin expiración es una fuga de memoria en cámara lenta.
- ✓ **Configura Maxmemory:** Usa allkeys-lru para cache. noeviction bloqueará escrituras al llenarse.
- ✓ **Usa Pipelines:** Agrupa comandos para reducir el round-trip de red.
- ✓ **Monitoreo Constante:** Vigila used_memory y el hit ratio.
- ✓ **Seguridad Estricta:** En producción requiere requirepass, TLS y ACLs.



El Error	El Impacto (Asesino Silencioso)	La Solución
KEYS * Usar comando KEYS para buscar.	 $O(N)$. Al ser single-threaded, bloquea toda la base de datos entera.	Usar SCAN con cursores.
Estampida de Cache (Stampede) Expiración simultánea de una clave muy consultada.	 Miles de requests fallan el cache al mismo milisegundo y asfixian a PostgreSQL.	TTL con jitter (tiempo aleatorio) o locks distribuidos (SET NX EX).
Megalitos JSON Guardar JSONs de >100KB en una sola clave String.	 Destruye el ancho de banda y la latencia de red.	Fragmentar o usar estructuras nativas (Hashes).

Ejercicio Práctico

1. Añade redis a tu docker-compose.yml.
2. Configura depends_on y healthchecks en tu App.
3. Identifica una query lenta (JOINS/Agregaciones) en tu Corte 3.
4. Implementa cache-aside con un TTL razonable (60-300s).
5. **Entregable:** Captura de logs demostrando caída en tiempo de respuesta, código del patrón y nota del hit ratio.

Redis no reemplaza a tu base de datos relacional. La **protege de lecturas masivas y acelera la experiencia de tus usuarios.**